May 2026

Porting RTEMS on the Versal AI Edge Board

Embedded Systems / Advanced Operating Systems Project Report

Alice Berta `alice.berta@mail.polimi.it`

Francesco Danese `francesco.danese@mail.polimi.it`

Supervisor:

Emilio Corigliano `emilio.corigliano@polimi.it`

Contents

1 Introduction

This project targets the AMD/Xilinx Versal AI Edge platform mounted on a Trenz TE0950 module. The primary objective is to demonstrate a functional real-time operating system (RTOS) running on the Real-Time Processing Unit (RPU) of the Versal device, specifically on the ARM Cortex-R5F cores operating in split mode. The RTOS used throughout this project is RTEMS (Real-Time Executive for Multiprocessor Systems). While a Board Support Package (BSP) for the Cortex-R5F split mode configuration already existed in the RTEMS codebase, no extensive or practical guide was available in the official RTEMS documentation. A further motivation for this work is therefore to contribute to the community by providing a clear, step-by-step guide on how to set up and use RTEMS on such a system.

The project is structured in two main parts. The first part covers the full toolchain setup and the porting process required to boot RTEMS on the RPU cores in split mode. The second part extends the work by driving a user LED connected to the Programmable Logic (PL) of the Versal device from an RTEMS application. The LED serves as a concrete proof that custom PL peripherals can be integrated and controlled by the RTOS: if a simple peripheral such as an LED can be successfully driven, the same approach can be applied to implement more complex PL-based systems.

The remainder of this report is organised as follows. Section 2 introduces the hardware and software stack used, providing a brief overview of the key components. Section 3 presents a concise, practical guide aimed at reproducing both goals of the project — booting RTEMS on the RPU in split mode and controlling the PL LED — in the most straightforward way possible. Section 4 offers a more extensive account of the full development process, documenting the failures encountered, the debugging sessions, and the decisions that led to the final working solution.

2 Hardware and Software Overview

2.1 Hardware Platform Overview

The hardware platform used throughout this project is the Trenz TE0950 system-on-module, which integrates an AMD/Xilinx Versal AI Edge device as its central component. The Versal family represents a heterogeneous compute architecture that goes significantly beyond a traditional SoC by combining, on a single die, a programmable logic fabric (PL) inherited from the FPGA lineage, a scalar processing subsystem (PS), and a dedicated AI Engine array. For the purposes of this project, the AI Engine array is not used, and the focus is entirely on the interaction between the PS and the PL.

Within the Processing System, two distinct processor complexes coexist with fundamentally different characteristics. The Application Processing Unit (APU) consists of dual ARM Cortex-A72 cores operating at high frequency, designed for general-purpose operating systems and throughput-oriented workloads; it belongs to the Full Power Domain (FPD) of the device. The Real-Time Processing Unit (RPU) consists of dual ARM Cortex-R5F cores, which are lower-frequency but deterministic processors with tightly coupled memories, lockstep capability, and an integrated Memory Protection Unit (MPU); it belongs to the Low Power Domain (LPD). The RPU is the target of this project.

The TE0950 module exposes several user-accessible peripherals relevant to this project. Importantly, the LED is active-low: writing a logical zero to the corresponding GPIO data register turns it on, while writing a logical one turns it off.

2.2 Software Stack Overview

The software stack involved in this project spans four distinct layers, each handled by a different tool, and understanding how they compose is essential before examining any individual step in detail. At the lowest layer, Vivado is used to create the hardware design: a block design is assembled graphically, connecting the CIPS (Control, Interfaces and Processing System) IP to the AXI NoC, then through an AXI SmartConnect to the AXI GPIO peripheral. Vivado performs synthesis, implementation, and bitstream generation, and finally exports the result as an XSA (Xilinx Support Archive) file, which is a container that bundles the hardware description and the bitstream together and serves as the handoff artifact between hardware and software tools.

At the second layer, Vitis Unified IDE consumes the XSA to create a platform component, which abstracts the hardware into a software-visible description of available peripherals, memory regions, and processor configurations. Vitis is also responsible for assembling the final bootable artifact: a PDI (Programmable Device Image), which is the binary loaded onto the Versal at boot time. A valid PDI for the Versal must contain at minimum the PLM (Platform Loader and Manager) firmware, the PSM (Platform Services Manager) firmware, the PL bitstream, and the application ELF binary targeted at the desired processor.

At the third layer, the RTEMS real-time operating system provides the execution environment for the application. RTEMS is built from source using a waf-based build system against the `arm/versal_rpu_lock_step` BSP, which provides the low-level startup code, interrupt controller initialization, and timer configuration for the Cortex-R5F in lock-step mode. The cross-compilation toolchain used is `arm-rtems7`, installed via the RTEMS Source Builder. The application itself is a single RTEMS task, the Init task.

At the fourth and final layer, JTAG connectivity provided by Vitis and the underlying XSDB (Xilinx System Debugger) tool is used both for programming the board and for debugging. The PDI can be loaded directly into the device over JTAG without writing to flash.

3 Practical Guide

This section provides a concise, self-contained guide for reproducing the two main goals of the project on a Trenz TE0950 board. The guide is intentionally divided into two independent parts, so that a reader who only needs to boot RTEMS on the RPU can stop after the first part, and a reader who wants to extend the setup to drive a PL LED can continue with the second.

The first part covers the full path from an empty Vivado project to a running RTEMS application: creating the hardware design in Vivado, building the RTEMS cross-compilation toolchain and BSP from source, and assembling the final bootable image in the Vitis Unified IDE. The second part extends this foundation: the existing Vivado design is modified to expose an AXI GPIO peripheral connected to the user LED, a new XSA is exported, and a small RTEMS blinking-LED application is built and deployed using the updated Vitis platform.

3.1 Working RTEMS Configuration

This subsection guides you through obtaining a working RTEMS setup on the Versal RPU. At the end of these steps, RTEMS will be running on the Cortex-R5F core 0 in split mode and printing output over UART.

3.1.1 Vivado Hardware Design

Before starting the block design, the TRENZ TE0950 board support files must be installed in Vivado. These files provide pre-configured IP settings, pin constraints, and interface definitions specific to the TE0950 module, and without them Vivado cannot resolve the board-level interfaces required by the CIPS IP. The following steps, drawn from the guides by Florent Werbrouck [4, 5], describe the procedure:

1. Download the reference design archive `TE0950-te0950-rd-vivado_2024.2-build_1_20250423075112.zip` from the Trenz Electronic downloads portal [3] and extract it to a local directory.
2. In Vivado, navigate to `Tools` > `Settings` > `Vivado Store` > `Board Repository` and add the path to the extracted folder.

Once the board repository is configured, the TE0950 reference design becomes visible in the board selection list when creating a new Vivado project.

The Vivado hardware design was created following the online guide by Adam Taylor [2], which walks through the construction of a Versal AI Edge block design step by step: instantiating the CIPS IP, connecting it to the AXI NoC, and wiring the remaining infrastructure required to export a valid XSA.

> ***Note for readers pursuing the LED Blink Extension (Section 3.2).*** *If the second goal of the project — driving the PL user LED — is also in scope, it is advisable to* pause *before executing the synthesis, implementation, and bitstream-generation steps in the guide above. Instead, first read Section 3.2, which describes the additional AXI GPIO block that must be added to the design. Completing the full Vivado run twice (once for the*

base design and once again after the LED modifications) is unnecessarily time-consuming, since synthesis and implementation on Versal can take considerable time. Incorporating the GPIO peripheral up front allows a single synthesis–implementation–bitstream cycle to produce an XSA that is valid for both goals.

3.1.2 Compiling RTEMS

The following guide was inspired by the official RTEMS Getting Started documentation [1], which provides the canonical instructions for setting up the RTEMS Source Builder and building the RTEMS kernel from source.

RTEMS Toolchain Initialisation Ensure the host machine has a native, working C, C++, and Python environment: the GCC compiler, GNU Make, Python 3, Git, and other essential build tools must all be available before proceeding.

Create the working directory and clone the required repositories:

```
mkdir -p $HOME/quick-start/src
cd $HOME/quick-start/src
git clone https://gitlab.rtems.org/rtems/rtos/rtems.git
git clone https://gitlab.rtems.org/rtems/tools/rtems-source-builder.git rsb
```

Move into the RSB `rtems` subdirectory and build the cross-compilation toolchain for 32-bit ARM (the architecture of the Versal RPU):

```
cd $HOME/quick-start/src/rsb/rtems
../source-builder/sb-set-builder --prefix=$HOME/quick-start/rtems/7 7/rtems-arm
```

This step compiles the full toolchain from source and may take up to one hour. Once it completes, add the newly built binaries to PATH:

```
export PATH=$HOME/quick-start/rtems/7/bin:$PATH
```

Switch to the RTEMS kernel repository and list the available BSPs:

```
cd $HOME/quick-start/src/rtems
./waf bsplist
```

The output should include several Versal-specific entries. The relevant BSPs for the RPU are:

- `arm/versal_rpu_lock_step`
- `arm/versal_rpu_split_0`
- `arm/versal_rpu_split_1`

For this project the `arm/versal_rpu_split_0` BSP is used.

BSP Configuration and Compilation Before building the RTEMS kernel, a `config.ini` file must be created in the RTEMS kernel repository directory to select the target BSP and tune its options. Navigate to the repository root and create the file:

```
cd $HOME/quick-start/src/rtems
cat > config.ini << 'EOF'
[arm/versal_rpu_split_0]
BUILD_TESTS = True
BSP_CONSOLE_MINOR = 1
VERSAL_MEMORY_NOCACHE_ORIGIN = 0x20000000
VERSAL_MEMORY_NOCACHE_LENGTH = 0x00010000
EOF
```

Each option in the file serves a specific purpose:

- `[arm/versal_rpu_split_0]` — selects the target BSP; `waf` will build exactly this configuration.
- `BUILD_TESTS = True` — instructs the build system to compile the RTEMS test suite alongside the kernel, which is useful for verifying the toolchain and BSP.
- `BSP_CONSOLE_MINOR = 1` — routes the RTEMS console to UART1 instead of the default UART0. The default selection (minor 0) caused communication issues in this setup, as described in Section 4.1.

- VERSAL_MEMORY_NOCACHE_ORIGIN — sets the base address of the non-cached memory region.
- VERSAL_MEMORY_NOCACHE_LENGTH — sets the size of that region to 64 KiB. Together, these two options define a contiguous non-cached window that proved helpful during debugging (see Section 4.1); they can be removed if cache coherency is not a concern for the target application.

With the configuration file in place, run `waf configure` to process it and set the installation prefix, then build the kernel and selected BSP. At the end install the RTEMS libraries and headers into the prefix directory:

```
cd $HOME/quick-start/src/rtems
./waf configure --prefix=$HOME/quick-start/rtems/7
./waf
./waf install
```

Create Your Application and Cross-Compile With the RTEMS toolchain and BSP installed, the next step is to create a minimal application, set up its build system, and cross-compile it against the installed BSP.

Create the application directory structure:

```
cd $HOME/quick-start
mkdir app
cd app
mkdir hello_world
cd hello_world
```

Download the `waf` build tool, make it executable, and initialise a Git repository with the `rtems_waf` helper as a submodule:

```
curl https://waf.io/waf-2.0.19 > waf
chmod +x waf
git init
git submodule add https://gitlab.rtems.org/rtems/tools/rtems_waf.git rtems_waf
```

Create the RTEMS configuration header `init.c`, which declares the kernel objects and drivers required by the application:

```
cat > init.c << 'EOF'
#define CONFIGURE_APPLICATION_NEEDS_CLOCK_DRIVER
#define CONFIGURE_APPLICATION_NEEDS_CONSOLE_DRIVER

#define CONFIGURE_UNLIMITED_OBJECTS
#define CONFIGURE_UNIFIED_WORK_AREAS

#define CONFIGURE_RTEMS_INIT_TASKS_TABLE

#define CONFIGURE_INIT

#include <rtems/confdefs.h>
EOF
```

Create the application source file `hello_world.c`, which defines the `Init` task — the RTEMS entry point equivalent to `main`:

```
cat > hello_world.c << 'EOF'
#include <rtems.h>
#include <stdio.h>

rtems_task Init(rtems_task_argument ignored)
{
    printf("Hello World!\n");
    rtems_task_exit();
}
EOF
```

Create the `wscript` build script, which instructs `rtems_waf` how to configure and link the application:

```
cat > wscript << 'EOF'
from rtems_waf import rtems

def init(ctx):
    rtems.init(ctx, version='7')

def bsp_configure(conf, arch_bsp):
    pass

def options(opt):
    rtems.options(opt)

def configure(conf):
    rtems.configure(conf)

def build(bld):
    rtems.build(bld)
    bld(features='c cprogram',
        target='hello_world.exe',
        source=['hello_world.c', 'init.c'])
EOF
```

Configure the application build against the installed RTEMS prefix and the chosen BSP:

```
./waf configure --rtems="$HOME/quick-start/rtems/7" \
                --rtems-bsp=arm/versal_rpu_split_0
```

Then build the application:

```
./waf
```

After a successful build, the compiled application is placed in `build/arm-rtems7-versal_rpu_split_0/`. The output file is named `hello_world.exe` but is in ELF format and is the binary that will be loaded onto the board in the next steps.

3.1.3 Vitis Workspace Setup

This subsection describes how to use the Vitis Unified IDE to ingest the XSA produced by Vivado, build the required firmware components, assemble a bootable PDI, and deploy it to the board over JTAG.

Creating the Workspace and Platform Component Launch the Vitis Unified IDE and, when prompted, select or create a workspace directory. Once the IDE is open, create a new platform component:

1. Navigate to *File* → *New Component* → *Platform*.
2. Assign a name to the platform (e.g. `versal_platform`) and choose the workspace as its location.
3. When asked for the hardware source, select *XSA file* and browse to the `.xsa` file exported from Vivado.
4. Leave the operating system set to *standalone* and the processor to *psv_cortexr5_0*; Vitis will derive the correct PLM and PSM firmware from the XSA automatically.
5. Confirm and let the IDE generate the platform.

Build the platform by clicking the *Build* button in the Flow panel (or right-clicking the platform component and selecting *Build*). This step compiles the PLM and PSM ELF binaries that are mandatory constituents of any Versal PDI.

Creating the Boot Image With the platform built, assemble the final PDI using the *Create Boot Image* wizard:

1. Navigate to *Vitis* → *Create Boot Image* (Versal).
2. Set the output PDI path to a convenient location.
3. Add the boot partitions in the following order:
    - **PLM ELF** — found under the platform build directory at `export/versal_platform/sw/versal_platform/boot/plm.elf`.
    - **PSM ELF** — located in the same directory as `psm_fw.elf`.

- **RTEMS application ELF** — the `hello_world.exe` built in the previous step, located in `$HOME/quick-start/app/hello_world/build/arm-rtems7-versal_rpu_split_0/`. Before adding it, rename the file from `hello_world.exe` to `hello_world.elf`; Vitis may reject or mishandle the `.exe` extension when assembling the PDI. Set the *Partition type* to *ELF* and the *Destination CPU* to *R5-0*.

4. Click *Create Image*; Vitis will produce the `.pdi` output file.

Programming the Board over JTAG Connect the TE0950 board via the JTAG/USB cable and ensure it is powered on. Load the PDI directly into the device without writing to flash:

1. Navigate to *Vitis* → *Program Device*.
2. Select the `.pdi` file produced in the previous step.
3. Click *Program*; Vitis hands the image to XSDB, which programs the device over JTAG.

Once programming completes, RTEMS boots on the Cortex-R5F core 0 and the `Hello World!` message is printed over UART1. Open a serial terminal at 115200 baud on the port corresponding to the board's UART1 to observe the output.

3.2 LED Blink Extension

This subsection extends the base RTEMS configuration to drive a user LED connected to the Programmable Logic of the Versal device. The LED is controlled by an AXI GPIO peripheral added to the Vivado block design, exposed to the RPU as a memory-mapped register, and toggled by a small RTEMS application running on the Cortex-R5F.

3.2.1 Vivado Hardware Design

This step modifies the Vivado block design produced in Section 3.1.1; that design must be completed first before proceeding here.

Background: memory-mapped I/O and the Versal interconnect The LED is controlled through *memory-mapped I/O* (MMIO): no direct wire connects the CPU to the pin; instead, the CPU performs ordinary load/store instructions to specific addresses, and dedicated hardware peripheral registers translate those writes into logic levels on a physical pin.

Step 1 — Enable `M_AXI_LPD` on the CIPS Double-click `versal_cips_0` and click *Next* through the CIPS wizard until reaching *PS-PMC* → *PS-PL Interfaces* (or *PS-PMC Interfaces*). Locate the *PS PL Interfaces* or *AXI Master Interfaces* section and enable `M_AXI_LPD`.

Configure the following options:

- *Data Width*: `32-bit` (sufficient for GPIO and avoids a width-conversion step).
- *AXI Master Clock Source*: *PL Clock 0* (`pl0_ref_clk`).

Click *Finish*/*OK*. Back in the block design, a new `M_AXI_LPD` port will appear on the CIPS block.

Step 2 — Add the AXI GPIO IP Right-click on an empty area of the block design canvas and select *Add IP*. Search for `AXI GPIO` and double-click the result to instantiate it. Double-click the newly added block to open its configuration dialog:

- *GPIO Width*: 1 (one bit is sufficient for a single LED).
- *All Outputs*: enable this checkbox so the direction register is hardwired to output and the pin cannot accidentally be driven as an input.
- *Default Output Value*: `0x1` (the TE0950 user LED D1 is active-low; initialising the output high keeps it off at reset).
- Disable *Enable Dual Channel* and *Enable Interrupt*.

Step 3 — Add the AXI SmartConnect IP Add a second IP: search for `AXI SmartConnect` and instantiate it. Configure it with: *Number of Slave Interfaces*: 1, *Number of Master Interfaces*: 1, *Number of Clocks*: 1. Leave all other settings at their defaults.

Step 4 — Wire the blocks Connect the following ports manually in the canvas:

- `versal_cips_0.M_AXI_LPD` → `smartconnect_0.S00_AXI`
- `smartconnect_0.M00_AXI` → `axi_gpio_0.S_AXI`
- `smartconnect_0.aclk` ← `versal_cips_0.pl0_ref_clk`
- `axi_gpio_0.s_axi_aclk` ← `versal_cips_0.pl0_ref_clk`
- `smartconnect_0.aresetn` ← `rst_versal_cips_0.peripheral_aresetn`
- `axi_gpio_0.s_axi_aresetn` ← `rst_versal_cips_0.peripheral_aresetn`

Step 5 — Expose the GPIO pin as an external port Right-click the GPIO output pin on `axi_gpio_0` and select *Make External*. Rename the resulting port to `gpio_rtl_0` in the *External Interface Properties* panel. This name will be referenced in the XDC constraints file in Step 7.

Step 6 — Address Editor (critical: 32-bit assignment for the RPU) Open the *Address Editor* tab. Expand the tree under `versal_cips_0` in `Network1` until you locate the entries for `M_AXI_LPD` (the RPU's 32-bit master view). It should be unassigned, if so click the `Assign all` button at the top.

Under `M_AXI_LPD`, find the row for `axi_gpio_0`: it should have *Master Base Address* set to `0x8000_0000` and *Range* set to 64K. This is the address the RTEMS driver will use to access the GPIO registers at runtime.

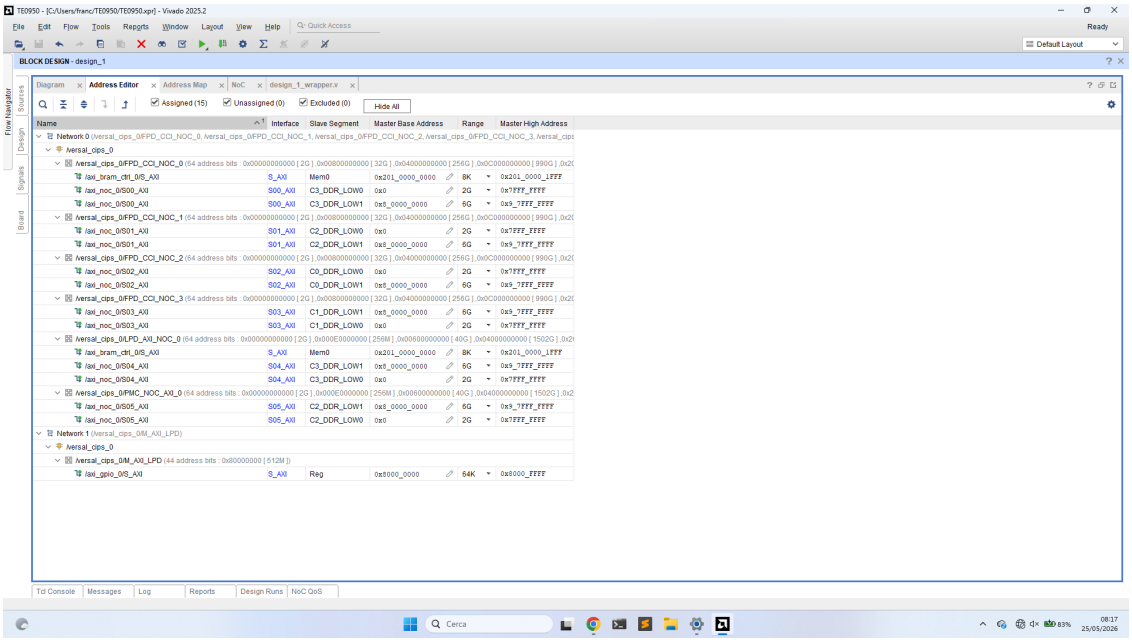

Figure 1: Vivado Address Editor: note the `0x80000000` base address assigned to `axi_gpio_0` under the `M_AXI_LPD` master view of the CIPS. This address falls within the 32-bit LPD address space and is directly reachable by the Cortex-R5F.

Step 7 — XDC pin constraints Create a constraints file (*Sources* → *Add Sources* → *Add or Create Constraints*) and add the following two lines, which assign the GPIO output to the physical LED pin on the TE0950:

```
set_property PACKAGE_PIN B12 [get_ports {gpio_rtl_0_tri_o[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gpio_rtl_0_tri_o[0]}]
```

The block design should look approximately as shown below.

Step 8 — Validate the design Press F6 or select *Tools* → *Validate Design*. The validation must complete with no critical errors.

Step 9 — Synthesis, implementation, and bitstream generation Run the full implementation flow: *Generate Bitstream* from the *Flow Navigator* will automatically trigger synthesis and implementation if they have not been run yet. On Versal, this flow can take a significant amount of time; no additional intervention is required once it is launched.

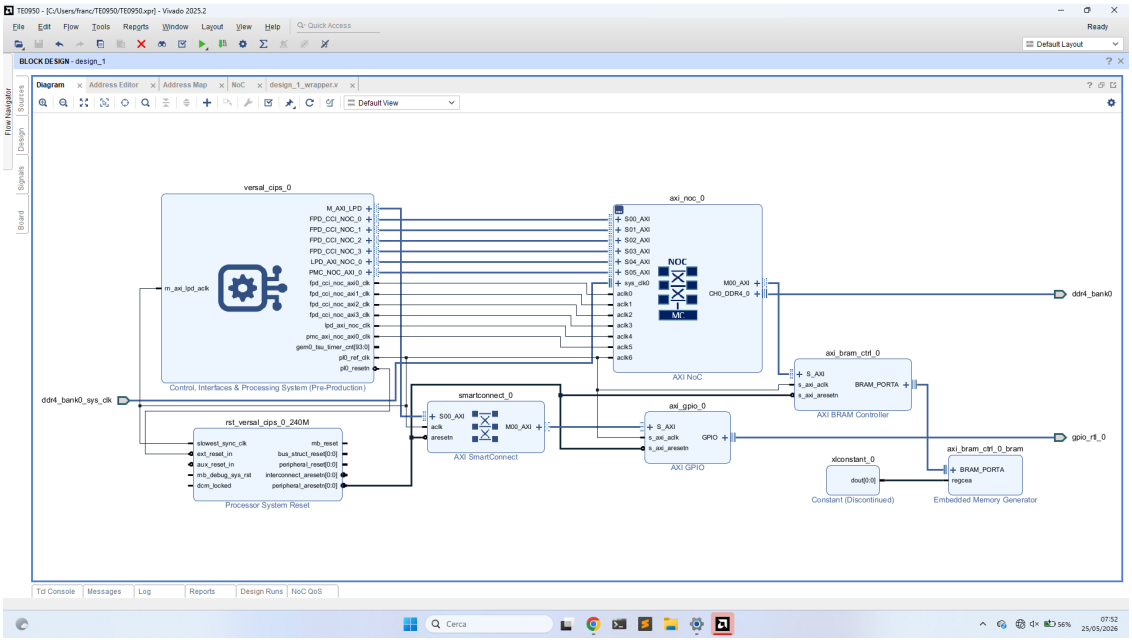

Figure 2: Vivado block design after completing Steps 1–7: the CIPS `M_AXI_LPD` port drives the AXI SmartConnect, which in turn connects to the AXI GPIO peripheral.

Step 10 — Export XSA with bitstream Once bitstream generation completes, export the hardware: *File* → *Export* → *Export Hardware*. In the wizard, select *Include bitstream* and save the `.xsa` file to a convenient path. This updated XSA will be imported into Vitis in the following subsection to rebuild the platform and deploy the LED application.

3.2.2 Building and Deploying the LED Application

The application build process mirrors the one described in Section 3.1.2 exactly: the same RTEMS toolchain, the same BSP (`arm/versal_rpu_split_0`), and the same `waf`-based workflow are reused. The only differences are the source files, the executable name, and the fact that the Vitis platform must be updated to consume the new XSA produced in the previous step before assembling the PDI.

The application directory structure is:

```
$HOME/quick-start/app/blinking_led/
  blinking_led.c    # application logic
  init.c            # RTEMS kernel configuration
  wscript           # waf build script
```

init.c The RTEMS configuration header is identical to the one used in the base application (see Section 3.1.2); copy it verbatim.

blinking_led.c This file contains the Init task that drives the LED. The GPIO DATA register is accessed through a single volatile pointer pinned to the address assigned in the Vivado Address Editor (`0x80000000`). Writing `0x0` to that address asserts the pin low, which turns the LED on (the TE0950 user LED is active-low); writing `0x1` drives it high and turns it off. A data synchronisation barrier (`dsb sy`) is issued after each write to ensure the store reaches the peripheral before the next sleep call. After an initial 5-second delay (to allow the system to settle after JTAG programming), the task toggles the LED on and off every 2 seconds for ten iterations, then exits.

```
#include <rtems.h>
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

#define USR_LED_ADDR 0x80000000
#define USR_LED (* (volatile uint32_t *) USR_LED_ADDR)

#define USR_LED_ON  0x0
#define USR_LED_OFF 0x1

#define MEMORY_BARRIER __asm__ volatile ("dsb sy" ::: "memory")

void turn_led_on()  { USR_LED = USR_LED_ON;  }
void turn_led_off() { USR_LED = USR_LED_OFF; }

rtems_task Init(rtems_task_argument ignored)
{
    sleep(5);
    printf("Starting...\n");

    for (volatile uint32_t i = 0; i < 10; i++) {
        if (i % 2 == 0)
            turn_led_on();
        else
            turn_led_off();

        MEMORY_BARRIER;
        sleep(2);
    }

    printf("...exiting.\n");
    rtems_task_exit();
}
```

wscript The `wscript` is identical in structure to the one in Section 3.1.2; adjust the target name and source list:

```
def build(bld):
    rtems.build(bld)
    bld(features='c cprogram',
        target='blinking_led.exe',
        source=['blinking_led.c', 'init.c'])
```

Configuring and building the application Configure and build exactly as before, pointing `waf` at the same RTEMS prefix and BSP:

```
cd $HOME/quick-start/app/blinking_led
./waf configure --rtems="$HOME/quick-start/rtems/7" \
                --rtems-bsp=arm/versal_rpu_split_0
./waf
```

The compiled binary `blinking_led.exe` is placed under `build/arm-rtems7-versal_rpu_split_0/`.

Updating the Vitis platform and deploying In Vitis, open the existing platform component (`versal_platform`) and update its hardware source to point to the new `.xsa` exported in the previous step; then rebuild the platform. Reassemble the PDI through *Create Boot Image* as described in Section 3.1.3, replacing the application ELF partition with `blinking_led.exe`; rename it to `blinking_led.elf` first, as Vitis may reject or mishandle the `.exe` extension when assembling the PDI. Keep the PLM and PSM ELF entries unchanged. Program the board with the new PDI over JTAG. After a 5-second pause the UART console will print `Starting...`, and the user LED D1 on the TE0950 will begin toggling every 2 seconds.

4 Development Process

This chapter documents the practical challenges encountered during development, together with the reasoning behind the choices made to address them. Its purpose is twofold: to give a transparent account of the decisions that shaped the final working configuration, and to serve as a reference for future developers facing similar issues on the same platform. The chapter is divided into two parts: the first covers the work required to establish that RTEMS was actually running on the board; the second addresses the challenges encountered when extending the

setup to drive the PL user LED.

4.1 Verifying RTEMS Execution

The XSA file used throughout the entire development process is the one produced in Section 3.1.1: a single hardware design was exported at the outset and reused in every subsequent step, including the bare-metal experiments described below.

The first practical obstacle was verifying whether RTEMS was running at all. Initial attempts produced no output on the serial terminal, making it impossible to distinguish between a complete boot failure and a misconfigured console driver. Because the team had no prior experience with the Vitis workflow, bare-metal applications were used as a stepping stone: they offer a simpler execution model, and establishing that bare-metal serial output worked correctly was a prerequisite for any further diagnosis.

Bare-Metal Hello World on a Single RPU Core As a first step, the group reproduced the most advanced working configuration of the previous group: a single bare-metal *Hello World* application running on the RPU in split mode. This served to build familiarity with the Vitis workflow and to confirm that the hardware design and toolchain were functional. The example project was already provided by Vitis, so it was only necessary to configure and program the board correctly.

Dual-Core Bare-Metal and the Linker Script Issue The team then moved to running two bare-metal *Hello World* applications simultaneously, one on each Cortex-R5F core. This is where the first significant problem was encountered: each application was supposed to print "Hello from processor *x*" (with *x* being 0 or 1), yet both cores produced the same output string. The initial hypothesis was that the Platform Loader and Manager (PLM), that is the Versal firmware module responsible for distributing executables to the processing units and programmable logic at boot time, had loaded one executable on top of the other, causing both RPU cores to execute the same code. This was partially confirmed by the Vitis debugger: stepping into the `printf` call on RPU 1, the string argument was observed to be `"Hello from processor 0"` rather than the expected `"Hello from processor 1"`, indicating that RPU 1 was executing RPU 0's code.

Suspicion fell on the linker scripts. Inspection revealed that both scripts specified the same load address for the `.text` section:

```
axi_noc_0_ddr_memory_DDR_LOW_0 : ORIGIN = 0x40000, LENGTH = 0x7ffc0000
```

Because both executables were mapped to the same base address, the PLM overwrote the first image with the second when assembling the boot image, leaving both cores pointing to the same code in memory. The fix was to assign non-overlapping memory regions: the first application's linker script was left unchanged, while the second was updated to:

```
axi_noc_0_ddr_memory_DDR_LOW_0 : ORIGIN = 0x10000000, LENGTH = 0x10000000
```

A shared-memory region was also added in anticipation of the next experiment:

```
shared_memory : ORIGIN = 0x20000000, LENGTH = 0x00010000
```

The complete linker scripts are reported in Appendix A (Listing 1 and 2). After applying the fix, both applications ran correctly and printed their respective strings.

Shared-Memory Validation Without RTEMS With dual-core bare-metal communication established, the team devised a test to determine whether RTEMS was running without relying on RTEMS serial output. The approach was as follows: an RTEMS task would write a known value to a shared memory location, while a bare-metal application running on the other RPU core would poll that address, detect the change, and print a confirmation to the console. This exploited the fact that bare-metal serial output had already been verified to work, thereby decoupling the question of whether RTEMS was executing from the question of whether the RTEMS console driver was configured correctly.

Before involving RTEMS, the same mechanism was validated using two bare-metal applications to confirm that the shared-memory communication itself was reliable. The reader and writer implementations are reported in

Appendix A (Listing 3 and 4). Memory barriers were added manually to ensure that values written by one core were not served from a stale cache line by the other. An alternative would have been to use the Xilinx library function that disables caching entirely; however, since that approach could not be easily replicated inside an RTEMS application, the memory barrier solution was preferred and proved sufficient.

Integrating RTEMS into the Shared-Memory Test With the bare-metal shared-memory communication verified, the team proceeded to integrate RTEMS. The writer application was ported to an RTEMS task (Listing 5 in Appendix A), and a boot image was assembled in Vitis containing both that ELF and the bare-metal reader ELF. After programming the board, however, the result matched what expected: no output was visible from the RTEMS side, and the only console activity was the polling message from the bare-metal reader. Initial suspicion fell on the RTEMS linker scripts, but adjusting them did not resolve the issue. Since the problem appeared to lie below the application level, the investigation escalated to lower-level debugging: rather than observing program flow alone, it became necessary to inspect registers and system state directly. The team therefore switched to `xsdb`, the command-line debugger integrated into the Vitis toolchain. A natural diagnostic approach at this point was to read the UART and timer registers directly via `xsdb`, as their state could distinguish between two failure modes: RTEMS halting before any meaningful execution, versus RTEMS running normally but producing no visible output.

Low-Level Debugging with XSDB Reading the UART registers via `xsdb` revealed that UART0 had not been initialised. Sampling the program counter (PC) repeatedly showed it frozen at the same address across all reads, confirming that the CPU was not making progress. Using `addr2line` to resolve that address to a source location pinpointed the exact point of failure: RTEMS was spinning inside the UART0 initialisation routine, waiting for the transmit FIFO to drain before enabling the peripheral. Because UART0 had never been enabled by the hardware, the FIFO could not become empty, and the spin-wait never terminated.

As a first step to confirm the root cause, the spin-wait was commented out so that RTEMS could complete its initialisation and proceed without a correctly configured UART. With that change in place, the bare-metal reader successfully detected the sentinel value written to shared memory by the RTEMS writer task, confirming that RTEMS itself was fully operational and that the only problem was the UART initialisation.

Restoring Serial Output Rather than debugging the high-level `printf` path immediately, the team first bypassed the C library entirely and wrote characters directly to the UART data register. This approach allowed the peripheral itself to be verified as reachable before involving the RTEMS console driver. Both UART0 (base address `0xFF000000`) and UART1 (base address `0xFF010000`) were tested using a `custom_printf` helper that polls the transmit-FIFO-full flag and writes one byte at a time; the implementation is reported in Appendix A (Listing 6). Direct writes to UART1 produced visible output, while the same test on UART0 did not.

Inspecting the RTEMS BSP source revealed the root cause: the BSP defaults the console to minor device 0, which maps to UART0. Because the Versal hardware does not initialise UART0, the driver cannot configure it and console output is silently lost. The fix was a single addition to `config.ini`:

```
BSP_CONSOLE_MINOR = 1
```

This redirects the RTEMS console to UART1. After recompiling RTEMS with this setting, the original application code using standard `printf` worked correctly without any further changes.

> ***Note.*** *UART0 could have been enabled as an alternative path: in Vivado, opening the CIPS configuration dialog and navigating to PS PMC → Peripherals → UART allows UART0 to be activated and assigned to its default MIO pin pair. Doing so would have allowed the BSP default (minor device 0) to work without any change to* `config.ini`*. The* `BSP_CONSOLE_MINOR = 1` *approach was retained because it required no hardware redesign and the XSA had already been finalised.*

4.2 LED Blink Debugging

The most significant challenges in this part of the project were encountered in the Vivado hardware design phase. While the base Vivado design followed an existing online guide, no equivalent resource was found for adding an AXI GPIO peripheral connected to the user LED, so the necessary block design modifications were worked out independently.

NoC Address Space Issue The first major obstacle arose when trying to route the GPIO peripheral through the AXI NoC. In the Vivado Address Editor, all addresses automatically assigned to the GPIO via the NoC fell above the 4 GB

boundary, placing them outside the 32-bit address space of the RPU. Since the Cortex-R5F cannot issue memory transactions to addresses above 0xFFFFFFFF, these assignments were unusable. The solution was to bypass the NoC master port entirely and connect the LPD AXI port of the CIPS directly to an AXI SmartConnect, which in turn drives the GPIO peripheral. This path stays within the 32-bit LPD address space and allows the address to be assigned at 0x80000000, reachable by the RPU.

Bare-Metal Validation Once the XSA was correctly exported with the GPIO peripheral mapped at 0x80000000, the design was first validated with a bare-metal application. The LED toggled as expected, confirming that the Vivado design, the XDC pin constraints, and the address assignment were all correct.

RTEMS Crash and MPU Fix Replacing the bare-metal application with the RTEMS blinking-LED application (Listing 7 in Appendix A) caused an immediate crash. The most likely cause was that the memory region containing the GPIO address was not covered by any MPU region in the RTEMS BSP: on the Cortex-R5F, any access to a region not explicitly enabled in the MPU triggers a fault, regardless of whether the address is physically valid. The fix was to add the following entry to the MPU table in bsps/arm/versal_rpu/start/bspstart.c:

```
{
  .begin = 0xFFE00000U,
  .end   = 0xFFEFFFFFU,
  .rasr  = ARMV7_MPU_RASR_XN |
           ARMV7_MPU_RASR_AP(0x3) |
           ARMV7_MPU_RASR_TEX_1_S(0x0) |
           ARMV7_MPU_RASR_SIZE(0x13) |
           ARMV7_MPU_RASR_ENABLE
},
```

This entry covers the 1 MB region 0xFFE00000–0xFFEFFFFF and configures it as Device memory (non-cacheable, non-bufferable) with full read/write access and the Execute Never flag set. After recompiling RTEMS with this addition and programming the board, the application ran correctly and the LED blinked as expected.

A Source Listings

The listings below reproduce the source files referenced in Section 4. Listings 1 and 2 are the linker scripts for the writer (RPU 0) and reader (RPU 1) bare-metal applications; the sole differences between them are the base address and size of the DDR region and the inclusion of the shared-memory region.

Listing 1 — Writer linker script (`lscript_writer.ld`, RPU 0)

```
/*****************************************************************************
* Copyright (C) 2023 - 2025 Advanced Micro Devices, Inc. All Rights Reserved.
* SPDX-License-Identifier: MIT
******************************************************************************/

_STACK_SIZE = DEFINED(_STACK_SIZE) ? _STACK_SIZE : 0x2000;
_HEAP_SIZE = DEFINED(_HEAP_SIZE) ? _HEAP_SIZE : 0x2000;

_ABORT_STACK_SIZE = DEFINED(_ABORT_STACK_SIZE) ? _ABORT_STACK_SIZE : 1024;
_SUPERVISOR_STACK_SIZE = DEFINED(_SUPERVISOR_STACK_SIZE) ? _SUPERVISOR_STACK_SIZE : 2048;
_IRQ_STACK_SIZE = DEFINED(_IRQ_STACK_SIZE) ? _IRQ_STACK_SIZE : 1024;
_FIQ_STACK_SIZE = DEFINED(_FIQ_STACK_SIZE) ? _FIQ_STACK_SIZE : 1024;
_UNDEF_STACK_SIZE = DEFINED(_UNDEF_STACK_SIZE) ? _UNDEF_STACK_SIZE : 1024;

MEMORY
{
    psv_r5_0_atcm_MEM_0 : ORIGIN = 0x0, LENGTH = 0x10000
    psv_r5_0_atcm_lockstep_MEM_0 : ORIGIN = 0xFFE10000, LENGTH = 0x10000
    psv_r5_0_btcm_MEM_0 : ORIGIN = 0x20000, LENGTH = 0x10000
    psv_r5_0_btcm_lockstep_MEM_0 : ORIGIN = 0xFFE30000, LENGTH = 0x10000
    psv_pmc_ram_psv_pmc_ram : ORIGIN = 0xF2000000, LENGTH = 0x20000
    psv_r5_0_data_cache_MEM_0 : ORIGIN = 0xFFE50000, LENGTH = 0x10000
    psv_r5_0_instruction_cache_MEM_0 : ORIGIN = 0xFFE40000, LENGTH = 0x10000
    psv_r5_1_data_cache_MEM_0 : ORIGIN = 0xFFED0000, LENGTH = 0x10000
    psv_r5_1_instruction_cache_MEM_0 : ORIGIN = 0xFFEC0000, LENGTH = 0x10000
    psv_r5_0_atcm_global_MEM_0 : ORIGIN = 0xFFE00000, LENGTH = 0x10000
    psv_r5_1_atcm_global_MEM_0 : ORIGIN = 0xFFE90000, LENGTH = 0x10000
    psv_r5_0_btcm_global_MEM_0 : ORIGIN = 0xFFE20000, LENGTH = 0x10000
    psv_r5_1_btcm_global_MEM_0 : ORIGIN = 0xFFEB0000, LENGTH = 0x10000
    axi_noc_0_ddr_memory_DDR_LOW_0 : ORIGIN = 0x40000, LENGTH = 0x7ffc0000
    versal_cips_0_pspmc_0_psv_ocm_ram_0_memory_0 : ORIGIN = 0xfffc0000, LENGTH = 0x40000
```

```
    shared_memory : ORIGIN = 0x20000000, LENGTH = 0x00010000
}

ENTRY(_boot)

SECTIONS
{
.vectors : {
   KEEP (*(.vectors))
   *(.boot)
} > psv_r5_0_atcm_MEM_0

.bootdata : {
   *(.bootdata)
} > psv_r5_0_atcm_MEM_0

.text : {
   *(.text)
   *(.text.*)
   *(.gnu.linkonce.t.*)
   *(.plt)
   *(.gnu_warning)
   *(.gcc_execpt_table)
   *(.glue_7)
   *(.glue_7t)
   *(.vfp11_veneer)
   *(.ARM.extab)
   *(.gnu.linkonce.armextab.*)
} > axi_noc_0_ddr_memory_DDR_LOW_0

.note.gnu.build-id : {
   KEEP (*(.note.gnu.build-id))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.init : {
   KEEP (*(.init))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.fini : {
   KEEP (*(.fini))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.interp : {
   KEEP (*(.interp))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.note-ABI-tag : {
   KEEP (*(.note-ABI-tag))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.rodata : {
   __rodata_start = .;
   *(.rodata)
   *(.rodata.*)
   *(.gnu.linkonce.r.*)
   __rodata_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.rodata1 : {
   __rodata1_start = .;
   *(.rodata1)
   *(.rodata1.*)
   __rodata1_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.sdata2 : {
   __sdata2_start = .;
   *(.sdata2)
   *(.sdata2.*)
   *(.gnu.linkonce.s2.*)
   __sdata2_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.sbss2 : {
   __sbss2_start = .;
   *(.sbss2)
   *(.sbss2.*)
   *(.gnu.linkonce.sb2.*)
   __sbss2_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.data : {
```

```
   __data_start = .;
   *(.data)
   *(.data.*)
   *(.gnu.linkonce.d.*)
   *(.jcr)
   *(.got)
   *(.got.plt)
   __data_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.data1 : {
   __data1_start = .;
   *(.data1)
   *(.data1.*)
   __data1_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.got : {
   *(.got)
} > axi_noc_0_ddr_memory_DDR_LOW_0

.ctors : {
   __CTOR_LIST__ = .;
   ___CTORS_LIST___ = .;
   KEEP (*crtbegin.o(.ctors))
   KEEP (*(EXCLUDE_FILE(*crtend.o) .ctors))
   KEEP (*(SORT(.ctors.*)))
   KEEP (*(.ctors))
   __CTOR_END__ = .;
   ___CTORS_END___ = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.dtors : {
   __DTOR_LIST__ = .;
   ___DTORS_LIST___ = .;
   KEEP (*crtbegin.o(.dtors))
   KEEP (*(EXCLUDE_FILE(*crtend.o) .dtors))
   KEEP (*(SORT(.dtors.*)))
   KEEP (*(.dtors))
   __DTOR_END__ = .;
   ___DTORS_END___ = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.fixup : {
   __fixup_start = .;
   *(.fixup)
   __fixup_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.eh_frame : {
   *(.eh_frame)
} > axi_noc_0_ddr_memory_DDR_LOW_0

.eh_framehdr : {
   __eh_framehdr_start = .;
   *(.eh_framehdr)
   __eh_framehdr_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.gcc_except_table : {
   *(.gcc_except_table)
} > axi_noc_0_ddr_memory_DDR_LOW_0

.mmu_tbl (ALIGN(16384)) : {
   __mmu_tbl_start = .;
   *(.mmu_tbl)
   __mmu_tbl_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.ARM.exidx : {
   __exidx_start = .;
   *(.ARM.exidx*)
   *(.gnu.linkonce.armexidix.*.*)
   __exidx_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.preinit_array : {
   __preinit_array_start = .;
   KEEP (*(SORT(.preinit_array.*)))
   KEEP (*(.preinit_array))
   __preinit_array_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0
```

```
.init_array : {
   __init_array_start = .;
   KEEP (*(SORT(.init_array.*)))
   KEEP (*(.init_array))
   __init_array_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.fini_array : {
   __fini_array_start = .;
   KEEP (*(SORT(.fini_array.*)))
   KEEP (*(.fini_array))
   __fini_array_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.drvcfg_sec : {
    . = ALIGN(8);
    __drvcfgsecdata_start = .;
    KEEP (*(.drvcfg_sec))
    __drvcfgsecdata_end = .;
    __drvcfgsecdata_size = __drvcfgsecdata_end - __drvcfgsecdata_start;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.ARM.attributes : {
   __ARM.attributes_start = .;
   *(.ARM.attributes)
   __ARM.attributes_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.sdata : {
   __sdata_start = .;
   *(.sdata)
   *(.sdata.*)
   *(.gnu.linkonce.s.*)
   __sdata_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.sbss (NOLOAD) : {
   __sbss_start = .;
   *(.sbss)
   *(.sbss.*)
   *(.gnu.linkonce.sb.*)
   __sbss_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.tdata : {
   __tdata_start = .;
   *(.tdata)
   *(.tdata.*)
   *(.gnu.linkonce.td.*)
   __tdata_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.tbss : {
   __tbss_start = .;
   *(.tbss)
   *(.tbss.*)
   *(.gnu.linkonce.tb.*)
   __tbss_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.bss (NOLOAD) : {
   . = ALIGN(4);
   __bss_start__ = .;
   *(.bss)
   *(.bss.*)
   *(.gnu.linkonce.b.*)
   *(COMMON)
   . = ALIGN(4);
   __bss_end__ = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

_SDA_BASE_ = __sdata_start + ((__sbss_end - __sdata_start) / 2);
_SDA2_BASE_ = __sdata2_start + ((__sbss2_end - __sdata2_start) / 2);

.heap (NOLOAD) : {
   . = ALIGN(16);
   _heap = .;
   HeapBase = .;
   _heap_start = .;
   . += _HEAP_SIZE;
   _heap_end = .;
```

```
   HeapLimit = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.stack (NOLOAD) : {
   . = ALIGN(16);
   _stack_end = .;
   . += _STACK_SIZE;
   _stack = .;
   __stack = _stack;
   . = ALIGN(16);
   _irq_stack_end = .;
   . += _IRQ_STACK_SIZE;
   __irq_stack = .;
   _supervisor_stack_end = .;
   . += _SUPERVISOR_STACK_SIZE;
   . = ALIGN(16);
   __supervisor_stack = .;
   _abort_stack_end = .;
   . += _ABORT_STACK_SIZE;
   . = ALIGN(16);
   __abort_stack = .;
   _fiq_stack_end = .;
   . += _FIQ_STACK_SIZE;
   . = ALIGN(16);
   __fiq_stack = .;
   _undef_stack_end = .;
   . += _UNDEF_STACK_SIZE;
   . = ALIGN(16);
   __undef_stack = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

end = .;
}
```

Listing 2 — Reader linker script (`lscript_reader.ld`, RPU 1)

```
/*****************************************************************************
* Copyright (C) 2023 - 2025 Advanced Micro Devices, Inc. All Rights Reserved.
* SPDX-License-Identifier: MIT
******************************************************************************/

_STACK_SIZE = DEFINED(_STACK_SIZE) ? _STACK_SIZE : 0x2000;
_HEAP_SIZE = DEFINED(_HEAP_SIZE) ? _HEAP_SIZE : 0x2000;

_ABORT_STACK_SIZE = DEFINED(_ABORT_STACK_SIZE) ? _ABORT_STACK_SIZE : 1024;
_SUPERVISOR_STACK_SIZE = DEFINED(_SUPERVISOR_STACK_SIZE) ? _SUPERVISOR_STACK_SIZE : 2048;
_IRQ_STACK_SIZE = DEFINED(_IRQ_STACK_SIZE) ? _IRQ_STACK_SIZE : 1024;
_FIQ_STACK_SIZE = DEFINED(_FIQ_STACK_SIZE) ? _FIQ_STACK_SIZE : 1024;
_UNDEF_STACK_SIZE = DEFINED(_UNDEF_STACK_SIZE) ? _UNDEF_STACK_SIZE : 1024;

MEMORY
{
    psv_r5_0_atcm_MEM_0 : ORIGIN = 0x0, LENGTH = 0x10000
    psv_r5_0_atcm_lockstep_MEM_0 : ORIGIN = 0xFFE10000, LENGTH = 0x10000
    psv_r5_0_btcm_MEM_0 : ORIGIN = 0x20000, LENGTH = 0x10000
    psv_r5_0_btcm_lockstep_MEM_0 : ORIGIN = 0xFFE30000, LENGTH = 0x10000
    psv_pmc_ram_psv_pmc_ram : ORIGIN = 0xF2000000, LENGTH = 0x20000
    psv_r5_0_data_cache_MEM_0 : ORIGIN = 0xFFE50000, LENGTH = 0x10000
    psv_r5_0_instruction_cache_MEM_0 : ORIGIN = 0xFFE40000, LENGTH = 0x10000
    psv_r5_1_data_cache_MEM_0 : ORIGIN = 0xFFED0000, LENGTH = 0x10000
    psv_r5_1_instruction_cache_MEM_0 : ORIGIN = 0xFFEC0000, LENGTH = 0x10000
    psv_r5_0_atcm_global_MEM_0 : ORIGIN = 0xFFE00000, LENGTH = 0x10000
    psv_r5_1_atcm_global_MEM_0 : ORIGIN = 0xFFE90000, LENGTH = 0x10000
    psv_r5_0_btcm_global_MEM_0 : ORIGIN = 0xFFE20000, LENGTH = 0x10000
    psv_r5_1_btcm_global_MEM_0 : ORIGIN = 0xFFEB0000, LENGTH = 0x10000
    axi_noc_0_ddr_memory_DDR_LOW_0 : ORIGIN = 0x10000000, LENGTH = 0x10000000
    versal_cips_0_pspmc_0_psv_ocm_ram_0_memory_0 : ORIGIN = 0xfffc0000, LENGTH = 0x40000
    shared_memory : ORIGIN = 0x20000000, LENGTH = 0x00010000
}

ENTRY(_boot)

SECTIONS
{
.vectors : {
   KEEP (*(.vectors))
   *(.boot)
} > psv_r5_0_atcm_MEM_0

.bootdata : {
```

```
   *(.bootdata)
} > psv_r5_0_atcm_MEM_0

.text : {
   *(.text)
   *(.text.*)
   *(.gnu.linkonce.t.*)
   *(.plt)
   *(.gnu_warning)
   *(.gcc_execpt_table)
   *(.glue_7)
   *(.glue_7t)
   *(.vfp11_veneer)
   *(.ARM.extab)
   *(.gnu.linkonce.armextab.*)
} > axi_noc_0_ddr_memory_DDR_LOW_0

.note.gnu.build-id : {
   KEEP (*(.note.gnu.build-id))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.init : {
   KEEP (*(.init))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.fini : {
   KEEP (*(.fini))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.interp : {
   KEEP (*(.interp))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.note-ABI-tag : {
   KEEP (*(.note-ABI-tag))
} > axi_noc_0_ddr_memory_DDR_LOW_0

.rodata : {
   __rodata_start = .;
   *(.rodata)
   *(.rodata.*)
   *(.gnu.linkonce.r.*)
   __rodata_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.rodata1 : {
   __rodata1_start = .;
   *(.rodata1)
   *(.rodata1.*)
   __rodata1_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.sdata2 : {
   __sdata2_start = .;
   *(.sdata2)
   *(.sdata2.*)
   *(.gnu.linkonce.s2.*)
   __sdata2_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.sbss2 : {
   __sbss2_start = .;
   *(.sbss2)
   *(.sbss2.*)
   *(.gnu.linkonce.sb2.*)
   __sbss2_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.data : {
   __data_start = .;
   *(.data)
   *(.data.*)
   *(.gnu.linkonce.d.*)
   *(.jcr)
   *(.got)
   *(.got.plt)
   __data_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.data1 : {
   __data1_start = .;
   *(.data1)
```

```
   *(.data1.*)
   __data1_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.got : {
   *(.got)
} > axi_noc_0_ddr_memory_DDR_LOW_0

.ctors : {
   __CTOR_LIST__ = .;
   ___CTORS_LIST___ = .;
   KEEP (*crtbegin.o(.ctors))
   KEEP (*(EXCLUDE_FILE(*crtend.o) .ctors))
   KEEP (*(SORT(.ctors.*)))
   KEEP (*(.ctors))
   __CTOR_END__ = .;
   ___CTORS_END___ = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.dtors : {
   __DTOR_LIST__ = .;
   ___DTORS_LIST___ = .;
   KEEP (*crtbegin.o(.dtors))
   KEEP (*(EXCLUDE_FILE(*crtend.o) .dtors))
   KEEP (*(SORT(.dtors.*)))
   KEEP (*(.dtors))
   __DTOR_END__ = .;
   ___DTORS_END___ = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.fixup : {
   __fixup_start = .;
   *(.fixup)
   __fixup_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.eh_frame : {
   *(.eh_frame)
} > axi_noc_0_ddr_memory_DDR_LOW_0

.eh_framehdr : {
   __eh_framehdr_start = .;
   *(.eh_framehdr)
   __eh_framehdr_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.gcc_except_table : {
   *(.gcc_except_table)
} > axi_noc_0_ddr_memory_DDR_LOW_0

.mmu_tbl (ALIGN(16384)) : {
   __mmu_tbl_start = .;
   *(.mmu_tbl)
   __mmu_tbl_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.ARM.exidx : {
   __exidx_start = .;
   *(.ARM.exidx*)
   *(.gnu.linkonce.armexidix.*.*)
   __exidx_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.preinit_array : {
   __preinit_array_start = .;
   KEEP (*(SORT(.preinit_array.*)))
   KEEP (*(.preinit_array))
   __preinit_array_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.init_array : {
   __init_array_start = .;
   KEEP (*(SORT(.init_array.*)))
   KEEP (*(.init_array))
   __init_array_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.fini_array : {
   __fini_array_start = .;
   KEEP (*(SORT(.fini_array.*)))
   KEEP (*(.fini_array))
   __fini_array_end = .;
```

```
} > axi_noc_0_ddr_memory_DDR_LOW_0

.drvcfg_sec : {
    . = ALIGN(8);
    __drvcfgsecdata_start = .;
    KEEP (*(.drvcfg_sec))
    __drvcfgsecdata_end = .;
    __drvcfgsecdata_size = __drvcfgsecdata_end - __drvcfgsecdata_start;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.ARM.attributes : {
   __ARM.attributes_start = .;
   *(.ARM.attributes)
   __ARM.attributes_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.sdata : {
   __sdata_start = .;
   *(.sdata)
   *(.sdata.*)
   *(.gnu.linkonce.s.*)
   __sdata_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.sbss (NOLOAD) : {
   __sbss_start = .;
   *(.sbss)
   *(.sbss.*)
   *(.gnu.linkonce.sb.*)
   __sbss_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.tdata : {
   __tdata_start = .;
   *(.tdata)
   *(.tdata.*)
   *(.gnu.linkonce.td.*)
   __tdata_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.tbss : {
   __tbss_start = .;
   *(.tbss)
   *(.tbss.*)
   *(.gnu.linkonce.tb.*)
   __tbss_end = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.bss (NOLOAD) : {
   . = ALIGN(4);
   __bss_start__ = .;
   *(.bss)
   *(.bss.*)
   *(.gnu.linkonce.b.*)
   *(COMMON)
   . = ALIGN(4);
   __bss_end__ = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

_SDA_BASE_ = __sdata_start + ((__sbss_end - __sdata_start) / 2);
_SDA2_BASE_ = __sdata2_start + ((__sbss2_end - __sdata2_start) / 2);

.heap (NOLOAD) : {
   . = ALIGN(16);
   _heap = .;
   HeapBase = .;
   _heap_start = .;
   . += _HEAP_SIZE;
   _heap_end = .;
   HeapLimit = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

.stack (NOLOAD) : {
   . = ALIGN(16);
   _stack_end = .;
   . += _STACK_SIZE;
   _stack = .;
   __stack = _stack;
   . = ALIGN(16);
   _irq_stack_end = .;
   . += _IRQ_STACK_SIZE;
   __irq_stack = .;
```

```
   _supervisor_stack_end = .;
   . += _SUPERVISOR_STACK_SIZE;
   . = ALIGN(16);
   __supervisor_stack = .;
   _abort_stack_end = .;
   . += _ABORT_STACK_SIZE;
   . = ALIGN(16);
   __abort_stack = .;
   _fiq_stack_end = .;
   . += _FIQ_STACK_SIZE;
   . = ALIGN(16);
   __fiq_stack = .;
   _undef_stack_end = .;
   . += _UNDEF_STACK_SIZE;
   . = ALIGN(16);
   __undef_stack = .;
} > axi_noc_0_ddr_memory_DDR_LOW_0

end = .;
}
```

Listing 3 — Bare-metal writer (`writer.c`, RPU 0)

```
#define SHARED_FLAG ((volatile uint32_t *) 0x20000000)

int main()
{
    init_platform();

    sleep(3);

    *SHARED_FLAG = 5;
    __asm__ volatile ("dsb sy" ::: "memory");

    if (*SHARED_FLAG == 5) {
        print("Writer: write completed\n");
    } else {
        print("Writer: write not completed\n");
    }

    cleanup_platform();
    return 0;
}
```

Listing 4 — Bare-metal reader (`reader.c`, RPU 1)

```
#define SHARED_FLAG ((volatile uint32_t *) 0x20000000)

int main()
{
    init_platform();

    sleep(3);

    int cycle = 0;

    __asm__ volatile ("dsb sy" ::: "memory");
    while (*SHARED_FLAG != 5) {
        cycle++;

        if (cycle % 5 == 0)
            printf("Reader: polling... cycle %d\n", cycle);

        sleep(1);
        __asm__ volatile ("dsb sy" ::: "memory");
    }

    __asm__ volatile ("dsb sy" ::: "memory");
    *SHARED_FLAG = 2;
    __asm__ volatile ("dsb sy" ::: "memory");

    printf("Reader: successful communication\n");

    cleanup_platform();
    return 0;
}
```

Listing 5 — RTEMS writer task (`rtems_init.c`)

```
#include <rtems.h>
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

#define SHARED_MEM_ADDR     0x20000000U
#define SHARED_FLAG         ((volatile uint32_t *)SHARED_MEM_ADDR)

#define WRITER_READY_VALUE  0x1U   /* written by RPU0 to signal RPU1 */

rtems_task Init(rtems_task_argument ignored)
{
    *SHARED_FLAG = 0U;
    __asm__ volatile ("dsb sy" ::: "memory");

    sleep(3);

    *SHARED_FLAG = WRITER_READY_VALUE;
    __asm__ volatile ("dsb sy" ::: "memory");

    if (*SHARED_FLAG == WRITER_READY_VALUE) {
        printf("[RPU_0] Write succeeded\n");
    } else {
        printf("[RPU_0] Write NOT succeeded\n");
    }

    rtems_task_exit();
}
```

Listing 6 — RTEMS writer with direct UART access (`rtems_init_uart.c`)

```
#include <rtems.h>
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>


#define SHARED_MEM_ADDR     0x20000000U
#define SHARED_FLAG         ((volatile uint32_t *)SHARED_MEM_ADDR)

#define WRITER_READY_VALUE  0x1U   /* written by RPU0 to signal RPU1 */

/* UART1 base: 0xFF010000  (UART0 base would be 0xFF000000) */
#define UART_DR  (*(volatile uint32_t *)0xFF010000)
#define UART_FR  (*(volatile uint32_t *)0xFF010018)

void custom_printf(char *str);

rtems_task Init(rtems_task_argument ignored)
{
    *SHARED_FLAG = 0U;
    __asm__ volatile ("dsb sy" ::: "memory");

    sleep(3);

    *SHARED_FLAG = WRITER_READY_VALUE;
    __asm__ volatile ("dsb sy" ::: "memory");

    if (*SHARED_FLAG == WRITER_READY_VALUE) {
        custom_printf("[RPU_0] Write succeeded\n");
    } else {
        custom_printf("[RPU_0] Write NOT succeeded\n");
    }

    rtems_task_exit();
}

void custom_printf(char *str) {
    for (uint32_t i = 0; i < strlen(str); i++) {
        while (UART_FR & (1 << 5)) { } /* wait while TX FIFO full */
        UART_DR = str[i];
        for (volatile int d = 0; d < 100000; d++) { }
    }
}
```

Listing 7 — RTEMS blinking-LED application (`blinking_led.c`)

```
#include <rtems.h>
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>

#define USR_LED_ADRR 0x80000000
#define USR_LED (* (volatile uint32_t *) USR_LED_ADRR)

#define USR_LED_ON 0x0
#define USR_LED_OFF 0x1

#define MEMORY_BARRIER __asm__ volatile ("dsb sy" ::: "memory")

void turn_led_on() {
  USR_LED = USR_LED_ON;
}

void turn_led_off() {
  USR_LED = USR_LED_OFF;
}

rtems_task Init(rtems_task_argument ignored) {

  sleep(5);

  printf("Starting...\n");

  for (volatile uint32_t i = 0; i < 10; i++) {
    if (i % 2 == 0)
      turn_led_on();
    else
      turn_led_off();

    MEMORY_BARRIER;

    sleep(2);
  }

  printf("...exiting.\n");

  rtems_task_exit();
}
```

References

[1] RTEMS Project. *RTEMS User Manual: Getting Started*. 2024. URL: https://docs.rtems.org/docs/main/user/start/index.html (visited on 05/18/2026).

[2] Adam Taylor. *MicroZed Chronicles: Creating a Versal AI Edge Design*. Adiuvo Engineering & Training Ltd. 2022. URL: https://www.adiuvoengineering.com/post/microzed-chronicles-creating-a-versal-ai-edge-design (visited on 05/18/2026).

[3] Trenz Electronic GmbH. *TE0950 Reference Design 2024.2 — Downloads*. 2025. URL: https://www.trenz-electronic.de/en/Downloads?path=Trenz_Electronic/Development_Boards/TE0950/Reference_Design/2024.2/test_board (visited on 05/25/2026).

[4] Florent Werbrouck. *01 — Getting Started with the Trenz TE0950 Board*. Hackster.io. 2023. URL: https://www.hackster.io/florent-werbrouck/01-getting-started-with-the-trenz-te0950-board-74c788 (visited on 05/25/2026).

[5] Florent Werbrouck. *02 — Minimal Baremetal System (No DDR) on Trenz TE0950 Board*. Hackster.io. 2023. URL: https://www.hackster.io/florent-werbrouck/02-minimal-baremetal-system-no-ddr-on-trenz-te0950-board-81e9ee (visited on 05/25/2026).